

CATANAGENTS: LONG-TERM PLANNING AGENTS FOR PLAYING SETTLERS OF CATAN

Dhruv Malpani Rudy Juarez-Ovallos Sharayu Janga Vayun Gupta

University of Illinois Urbana-Champaign

dhruvm6@illinois.edu rjuare8@illinois.edu

sjanga2@illinois.edu vayung2@illinois.edu

1 INTRODUCTION

Large language models show strong performance on reasoning and tool-use tasks, but their reliability in long-horizon, multi-step strategic environments remains unclear. Settlers of Catan is a useful stress test: strong play requires planning under uncertainty, partial observability, stochastic dice outcomes, changing board geometry, and multi-agent negotiation. Winning is not a matter of one good move—agents must balance expansion, hand efficiency, opponent blocking, development-card timing, and trade policy across many turns. Our companion benchmark argues that outcome-only evaluation is especially weak here, since a strategically sound move can still lose to dice variance while a brittle sequence can win if not punished.

Existing agents are insufficient in this setting for two reasons. First, single-stream ReAct-style agents overload one prompt with parsing full state, long-term planning, threat detection, negotiation, and legal action selection; in our early prototypes this produced hallucinated actions, lost context, weak opponent modeling, and high latency. Second, raw environment state is too noisy for stable decision making in a socket-driven multiplayer game that must also respond to asynchronous trade offers. Prior work reinforces this: LLMs remain brittle in strategic games requiring adaptation and consistency (Yao et al., 2022; Vidler and Walsh, 2025).

We propose a role-specialized multi-agent Catan system with four cooperating agents—**Strategy**, **Development**, **Trading**, and **Risk**—coordinated through a shared scratchpad. Strategy is the orchestrator: it updates shared state, invokes Risk for deterministic probability analysis and threat summaries, produces a turn-level plan, and delegates build, discard, robber, and trade decisions. Unlike generic multi-agent chat, communication follows an explicit code-enforced topology, each agent has a restricted tool subset, and Risk combines exact game math with a lightweight LLM narrative rather than free-form language reasoning. The design is intended to reduce prompt overload, improve legality, and persist strategic intent across turns (Belle et al., 2025).

Evaluation is in progress at submission time: the benchmark environment, scratchpad, and Strategy and Trading agents are implemented; preliminary runs cover the setup phase, but full-game evaluation is blocked by incomplete Development-agent functionality. This draft focuses on agent design and the evaluation plan; empirical results are forthcoming. The remainder of the paper is organized as follows: Section 2 reviews related agent systems, Section 3 describes the architecture, Section 4 the experimental setup, Section 5 preliminary results, and Sections 6–7 discuss limitations and conclusions.

2 RELATED WORK

Our work is most directly related to **ReAct**, which interleaves reasoning and action so a single LLM can plan while interacting with an environment (Yao et al., 2022); it is our primary baseline. We depart from ReAct by decomposing decision making across specialized roles and constraining tool access per role, which reduces illegal actions and makes failures easier to trace. More broadly, our design is related to multi-agent LLM frameworks such as **CAMEL** and **AutoGen**, which use role specialization and inter-agent communication for complex tasks (Li et al., 2023; Wu et al., 2023). Unlike those general-purpose systems, our architecture is domain-specific: each agent has a fixed Catan role and communicates through a typed, scratchpad-mediated, topology-restricted protocol. **Voyager** motivates persistent intermediate structure for long-horizon tasks (Wang et al., 2023); we

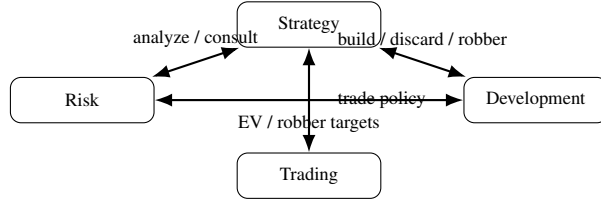


Figure 1: Communication topology. Edges are the only allowed channels; `BaseAgent` raises `PermissionError` on any unauthorized call. Trading is isolated so all strategic input flows through Strategy’s `TradePolicy`.

adopt a tightly scoped variant that stores strategic state, trade context, and risk summaries rather than executable skills. The closest domain work is **Agents of Change** (Belle et al., 2025), which studies self-evolving LLM agents for Catan; we instead target reliable in-game coordination and benchmarked decision quality during live play, complementing rather than replacing that line of work. Recent evidence that LLMs remain brittle in strategic games (Vidler and Walsh, 2025) further motivates our hybrid math-plus-LLM design.

3 AGENT DESIGN

3.1 ARCHITECTURE OVERVIEW

CatanAgents is a decentralized cooperative system of four specialized GPT-4o agents: the **Strategy** orchestrator and the **Risk**, **Development**, and **Trading** executors. All persistent state flows through a thread-safe *Scratchpad*, and inter-agent control flow is restricted to the topology in Figure 1, enforced at runtime by an `ALLOWED_CHANNELS` whitelist in `BaseAgent`. Per turn, Strategy polls the socket, writes state to the *Scratchpad*, invokes Risk for deterministic probability analysis, asks GPT-4o for a structured plan, and delegates tactical phases to Trading and Development.

Five principles govern the design: (1) *separation of concerns*, each agent owns one cognitive domain; (2) *hierarchical coordination*, Strategy drives a typed control loop; (3) *hybrid reasoning*, deterministic math for probability-critical quantities and LLMs only for ambiguous decisions; (4) *graceful degradation*, every LLM call has a deterministic fallback; and (5) *thread safety*, a single `threading.Lock` on the *Scratchpad* serializes the background socket listener and the main agent loop.

3.2 SHARED INFRASTRUCTURE

Scratchpad. The *Scratchpad* is the only mechanism for cross-agent state. It exposes nine regions, written with typed Python dataclasses rather than free-form dicts. `game_state`, `processed_state`, and `state_json` hold raw, structured, and LLM-optimized views of the game (all written by Strategy). `risk_analysis` (Risk) stores a `RiskAnalysis` with expected income, threat rankings, robber EVs, and a narrative. `strategy_plan` (Strategy) holds a `StrategyPlan` with goals, build queue, and `TradePolicy`. `trade_state` (Trading) tracks per-opponent reputation, pending offers, and trade history. `build_log` (Development) and `action_log` (all) are append-only audit trails; `inter_agent_messages` is a typed message queue. Because every region is a dataclass, turns are deterministically replayable from the logs.

Tool registry. A unified `ToolRegistry` exposes 22 game actions and queries. Each tool carries (i) an OpenAI JSON schema used directly for GPT-4o function-calling, (ii) a phase filter (`roll`, `main`, `discard`, `robber`, `setup`, `specialBuild`), and (iii) an agent filter. Role-scoping is strict: `move_robber` is Development-only; `bank_trade`, `propose_trade`, `respond_to_trade` are Trading-only. The registry computes the visible subset at prompt-construction time, dropping roughly half the tool descriptors from every agent-specific context.

3.3 PER-AGENT DESIGN

Strategy owns the game loop, polling state every 150–250 ms and calling `risk.analyze()` and `risk.consult(question)` each turn. It builds a GPT-4o prompt from the `state_json`, the `RiskAnalysis`, the `Risk` consultant’s answer, the previous plan, recent messages, and ranked building spots, receiving back a `StrategyPlan` JSON with `long_term_goal`, `short_term_goals`, `priority_resources`, `build_queue`, `trade_policy`, `should_trade_first`, `risk_tolerance`, and `reasoning`. It then delegates by phase: `main` runs `trading.proactive_trade()` then `development.execute_build_queue()`; `discard` and `robber` are forwarded to `Development`; off-turn, `Strategy` watches for incoming offers and wakes `trading.respond_to_offer()`. **Risk** is an interactive consultant with three modes—`analyze()` (runs the math in Section 3.5 plus a GPT-4o narrative), `consult(question)` (answers a turn-relevant question selected from a small decision table), and `assess_plan(plan)` (critiques a draft plan)—all with deterministic template fallbacks.

Development executes `StrategyPlan.build_queue` in priority order via registry tools; failures are skipped and logged rather than re-planned, because the server is the authority on legality. For `discard` on 7-rolls, GPT-4o selects cards via `discard_cards` using `Strategy`’s `priority_resources` (up to two repair rounds, then a greedy non-priority fallback). For `robber`, `Development` calls `risk.get_robber_targets()` for hexes ranked by net damage, optionally lets GPT-4o override, and falls back to an adjacency-count heuristic. **Trading** runs in two modes: *proactive* (our turn) runs a 6-step ReAct loop over trading tools (`bank_trade`, `propose_trade`, `counter_trade`, `cancel_trade`, `get_trade_options`, `get_trade_offer_status`), falling back to a bank trade then a 1-for-1 proposal; *reactive* (off-turn) deduplicates incoming offers by signature hash and runs a 2-step loop restricted to `respond_to_trade/counter_trade`, falling back to the deterministic score in Section 3.5. A per-opponent reputation in $[-1, 1]$ (+0.05 per accept, −0.03 per decline) biases proposal targeting; opponents at ≥ 8 VP are blocked from proposals entirely.

3.4 TOKEN-EFFICIENT STATE SUMMARY

Raw `getPlayerView` contains hundreds of hex, vertex, and edge entries that would dominate an LLM prompt. The Scratchpad emits a compact summary via `to_state_json()` with seven regions—`meta` (phase, turn, dice), `scoreboard` (VP, per-opponent cards/knights/road length), `my_state` (resources, dev cards, pieces left, trade ratios, buildings), `board_signals` (robber hex, longest road, largest army), `risk_summary` (expected income, top threat, narrative), `active_trade`, and `recent_delta`—which every agent prompt consumes. The summary gives consistency across agents on a given turn and is roughly $10\times$ smaller than the raw payload, keeping multi-step ReAct loops within the GPT-4o context budget.

3.5 PROBABILITY AND DECISION MODEL

All probability-critical decisions use a deterministic backend. For a vertex v with adjacent hexes $\text{adj}(v)$, expected per-turn production is

$$E[\text{income}_v] = \sum_{H \in \text{adj}(v)} \frac{\text{pips}(H)}{36} \cdot m \cdot \mathbf{1}[H \neq \text{desert, robber}], \quad (1)$$

with $m = 2$ for cities and $m = 1$ for settlements. Opponent threat score is $\text{score} = 10 \text{ VP} + 8r + 3d + 0.5c + b_{\text{road}} + b_{\text{army}} + 5a$, where r is income, d is dev cards, c is total cards, a is achievements, and $b_{\text{road}}, b_{\text{army}}$ are proximity bonuses; threats are *critical* when $\text{VP} \geq 8$ or $\text{score} \geq 75$. Settlement placements use a scarcity-weighted variant of Eq. 1, replacing m with a weight $w(\text{res}(H)) \in \{2.0, 1.5, 1.0, 0.6\}$ for hand counts $\{0, 1, 2-3, 4+\}$, plus a small diversity bonus for unseen resources. Trade evaluation produces a score in $[0, 1]$ starting at 0.5 with +0.25 per desperately-needed resource received and −0.20 per such resource given; opponents at $\text{VP} \geq 8$ incur −0.35. Counter-offers are built automatically in the near-threshold window $[\text{min_accept} - 0.2, \text{min_accept}]$. A per-player win probability combines VP, income, dev cards, and achievement proximity with weights (0.50, 0.20, 0.15, 0.15), normalized across players.

3.6 PROMPTING, ERRORS, AND TRADE-OFFS

Prompts live in per-agent `prompts.py` files and are composed from Scratchpad views. Strategy’s system prompt demands a single `StrategyPlan` JSON with no other text; Risk uses three prompts (consultant, plan-assessor, narrative); Development and Trading use task-scoped prompts with disjoint tool subsets. Tool-calling follows a bounded ReAct pattern (2 steps for discard/robber, 6 for proactive trading, 2 for reactive); malformed tool calls trigger a “repair” message until the cap, after which the deterministic fallback runs. Error handling is layered: OpenAI 429s use exponential backoff up to 90 s; illegal server actions are skipped rather than re-planned; the `threading.Lock` prevents write interleaving; and outgoing offers are capped by a 60 s polling loop that auto-cancels and falls back to a bank trade. We rejected three alternatives for principled reasons: a single ReAct agent suffers from prompt overload (the failure mode we observed in early prototypes); a fully LLM-driven pipeline (Risk doing math) was rejected because probability-critical quantities must be exact; and unconstrained multi-agent chat was rejected because it lets Trading gather data that contradicts Strategy. We pay for this in higher per-turn LLM usage, in exchange for legality, interpretability, and replayability.

4 EXPERIMENTAL SETUP

Benchmark. We evaluate on **CatanBench**, our companion benchmark for strategic reasoning in Settlers of Catan. It contains 20 tasks spanning short-term tactical choices, medium-horizon resource and build prioritization, and long-term strategic commitments, and it scores the *quality* of decisions during live play rather than final game outcomes alone. The benchmark is described fully in our benchmark paper; we summarize only the parts relevant to agent evaluation.

Baselines. We compare against two systems. *Single-Prompt ReAct*: one LLM with the full tool interface handles all reasoning and action execution alone—the most direct test of whether role specialization improves over a standard tool-using design. *Rule-Based LLM*: the same environment interface but guided by hand-crafted Catan heuristics, providing a lower-structure baseline.

Metrics. Gameplay metrics are win rate, average final victory points, and average rounds to win; reasoning metrics are task success rate and average task score across the 20 tasks; operational metrics are average per-turn latency, illegal-move rate, and retry rate. Overall performance is summarized with a weighted aggregate over normalized win rate, VP, rounds-to-win, task score, and latency; we report means, standard deviations, and confidence intervals across trials.

Protocol. All agents run under identical seeds and environment settings. We plan approximately **20–30 full-game trials** per system plus the full benchmark task set, across varied seat positions and map configurations. Each run logs structured traces of state transitions, chosen actions, legality, latency, token usage, API cost, and benchmark scores. Because full-game evaluation is still running at submission time, the final version will report exact model versions and trial counts.

5 RESULTS

Full-game evaluation is still in progress at the time of submission. Preliminary runs confirm that the system-level plumbing works end-to-end: the setup-phase heuristic produces legal opening placements on every seed we have tested, the scratchpad-based turn loop executes without deadlocks under concurrent socket updates, and Risk’s deterministic functions return consistent expected-income and threat scores on saved game states. The full cross-agent comparison against the ReAct and rule-based baselines is pending completion of the Development agent’s build-queue executor, which is the last module blocking end-to-end games.

Table 1 shows the headline comparison we will populate in the final version of this paper. Numbers are intentionally left as TBD; we report the table schema here so that our evaluation plan is concrete and reproducible.

Table 1: Planned headline comparison. Final numbers are pending full-game evaluation.

Agent	Win rate	Avg VP	Rounds / win	Task score	Illegal move %
Single-Prompt ReAct	TBD	TBD	TBD	TBD	TBD
Rule-Based LLM	TBD	TBD	TBD	TBD	TBD
CatanAgents (ours)	TBD	TBD	TBD	TBD	TBD

6 DISCUSSION

What works. Role-scoped tool sets substantially reduce the prompt surface each agent sees and in early testing eliminated a class of illegal actions seen with the single-ReAct prototype (e.g., Trading attempting `move_robber`, or Development proposing trades out of phase). Typed Scratchpad dataclasses make plans and risk analyses inspectable turn-by-turn, and because every LLM call has a deterministic fallback, the system has never stalled when OpenAI is unavailable—it runs on heuristics and logs the degradation. The `state_json` summary keeps prompts consistent and short enough that multi-step ReAct loops stay within context without truncation.

What does not work yet and limitations. The largest gap is Development’s end-to-end build-queue executor, which reliably handles discard and robber but still fails on a subset of city-upgrade and road-building items—the current blocker for full-game evaluation. Strategy’s LLM-generated queues occasionally reference vertex keys that opponents claim before execution; skip-and-continue hides this from the server but wastes queue slots. Proactive ReAct trading is also expensive: a 6-step GPT-4o loop can dominate per-turn cost while often matching the deterministic heuristic, suggesting the LLM should be invoked only when the heuristic is uncertain. The design assumes standard 4-player Catan and does not model expansions or cross-game self-improvement in the style of Wang et al. (2023) or Belle et al. (2025). GPT-4o cost per full game is nontrivial; our fallbacks are deliberately structured to make swapping in a smaller local model mechanical.

7 CONCLUSION

We have described CatanAgents, a role-specialized four-agent LLM system for Settlers of Catan that combines deterministic probability analysis with LLM-driven planning through a typed shared Scratchpad and a strict, code-enforced communication topology. The design contributes a concrete blueprint for decomposing a long-horizon strategic game across LLM agents while keeping probability-critical decisions exact and making strategic intent persistent across turns. Immediate future work is to complete the Development executor and run the full CatanBench evaluation; longer-term, we plan to add a cross-game self-improvement loop over the action log, swap in smaller local models behind the existing heuristic fallbacks, and extend the framework to Catan variants and larger player counts.

REFERENCES

- Nikolas Belle, Dakota Barnes, Alfonso Amayuelas, Ivan Bercovich, Xin Eric Wang, and William Wang. Agents of Change: Self-Evolving LLM Agents for Strategic Planning. *arXiv preprint arXiv:2506.04651*, 2025.
- Guohao Li, Hasan Abed Al Kader Hammoud, Hani Itani, Dmitrii Khizbullin, and Bernard Ghanem. CAMEL: Communicative Agents for “Mind” Exploration of Large Language Model Society. *Advances in Neural Information Processing Systems*, 2023.
- Alicia Vidler and Toby Walsh. Playing Games with Large Language Models: Randomness and Strategy. *arXiv preprint arXiv:2503.02582*, 2025.
- Guanzhi Wang, Yuqi Xie, Yunfan Jiang, Ajay Mandlekar, Chaowei Xiao, Yuke Zhu, Linxi Fan, and Anima Anandkumar. Voyager: An Open-Ended Embodied Agent with Large Language Models. *arXiv preprint arXiv:2305.16291*, 2023.

Qingyun Wu, Gagan Bansal, Jieyu Zhang, Yiran Wu, Beibin Li, Erkang Zhu, Li Jiang, Xiaoyun Zhang, Shaokun Zhang, Jiale Liu, Ahmed Hassan Awadallah, Ryen W. White, Doug Burger, and Chi Wang. AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation. *arXiv preprint arXiv:2308.08155*, 2023.

Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing Reasoning and Acting in Language Models. *arXiv preprint arXiv:2210.03629*, 2022.